

CO2 - AT3 -SCENARIO BASED QUESTIONS

Course Code: ITA0609

Course Title: Machine Learning

Student Name: S.Swagat

1. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model.

1.Sol:- Backpropagation Algorithm in a Multilayer Perceptron (MLP) for Binary Classification

Introduction

A Multilayer Perceptron (MLP) is a type of Artificial Neural Network (ANN) used to solve classification problems. It consists of three layers: an input layer, one or more hidden layers, and an output layer. In binary classification, the output belongs to one of two classes, such as Yes/No, 0/1, or Placed/Not Placed.

The backpropagation algorithm is a supervised learning algorithm used to train an MLP. It works by comparing the predicted output with the actual output, calculating the prediction error, and updating the weights to minimise this error. The process is repeated for several epochs until the model achieves good prediction accuracy.

Dataset: Student Placement Prediction

The following dataset is used to predict whether a student will be placed based on Aptitude Score, Communication Skill, and Programming Skill.

Example	Aptitude Score	Communication Skill	Programming Skill	Actual Output (Placed)	Predicted Output
1	85	High	High	Yes	Yes
2	60	Medium	High	Yes	No
3	45	Low	Medium	No	No
4	90	High	High	Yes	Yes
5	50	Medium	Low	No	Yes

Input Features

- Aptitude Score
- Communication Skill
- Programming Skill

Output Classes

- **Yes (1)** = Student is Placed
- **No (0)** = Student is Not Placed

For explanation, consider **Example 2**.

- Aptitude Score = **60**
- Communication Skill = **Medium**
- Programming Skill = **High**
- Actual Output = **Yes (1)**

Architecture of the MLP

Input Layer

Aptitude Score

Communication Skill

Programming Skill



Hidden Layer

(H1 and H2)



Output Layer

Placed (Yes / No)

Step 1: Forward Pass

Initially, the neural network assigns random weights to all connections.

Initial Weights

Input → Hidden Layer

$$w1 = 0.30$$

$$w2 = 0.40$$

$$w3 = 0.20$$

$$w4 = 0.50$$

$$w5 = 0.35$$

$$w6 = 0.45$$

Hidden → Output Layer

$$w7 = 0.60$$

$$w8 = 0.70$$

Each hidden neuron computes a weighted sum.

Weighted Sum

$$Z = (x1 \times w1) + (x2 \times w2) + (x3 \times w3) + b$$

where,

- $x1$ = Aptitude Score
- $x2$ = Communication Skill
- $x3$ = Programming Skill
- w = Weight
- b = Bias

The weighted sum is passed through the Sigmoid activation function.

Sigmoid Activation Function

$$\text{Output} = 1 / (1 + e^{(-Z)})$$

Suppose the hidden layer produces

$$\text{Hidden Neuron H1} = 0.82$$

$$\text{Hidden Neuron H2} = 0.79$$

The output neuron receives these values and predicts

$$\text{Predicted Output} = 0.72$$

The target value is

$$\text{Actual Output} = 1$$

Since the prediction is not equal to the target, an error is generated.

Step 2: Error Calculation

The prediction error is calculated as

$$\text{Error} = \text{Target} - \text{Predicted Output}$$

Substituting the values,

$$\text{Error} = 1 - 0.72$$

$$\text{Error} = 0.28$$

The Mean Squared Error (MSE) is

$$\text{MSE} = (1/2) \times (\text{Target} - \text{Predicted Output})^2$$

Calculation

$$\text{MSE} = (1/2) \times (1 - 0.72)^2$$

$$= (1/2) \times (0.28)^2$$

$$= (1/2) \times 0.0784$$

$$= 0.0392$$

The objective of training is to reduce this error as much as possible.

Step 3: Backward Propagation

During backpropagation, the error is propagated from the output layer to the hidden layer. The network calculates how much each weight contributed to the error and updates the weights.

Weight Update Formula

$$\text{New Weight} = \text{Old Weight} + (\text{Learning Rate} \times \text{Error} \times \text{Input})$$

or

$$w(\text{new}) = w(\text{old}) + \eta \times \text{Error} \times \text{Input}$$

where

- η = Learning Rate

Assume

$$\text{Old Weight} = 0.60$$

$$\text{Learning Rate} = 0.10$$

$$\text{Input} = 0.82$$

$$\text{Error} = 0.28$$

Weight update

New Weight

$$= 0.60 + (0.10 \times 0.28 \times 0.82)$$

$$= 0.60 + 0.02296$$

$$= 0.62296$$

$$\approx 0.623$$

Similarly, all the remaining weights are updated.

After several epochs, the prediction improves.

$$\text{Epoch 1} \rightarrow 0.72$$

Epoch 2 → 0.81

Epoch 3 → 0.89

Epoch 4 → 0.96

Epoch 5 → 0.99

The error gradually decreases, and the network learns the correct pattern.

Influence of Learning Rate

The learning rate determines how much the weights are updated during training.

Small Learning Rate ($\eta = 0.01$)

- Slow learning
- Stable convergence
- Requires many epochs

Large Learning Rate ($\eta = 1$)

- Faster learning
- May overshoot the minimum error
- Training becomes unstable

Moderate Learning Rate ($\eta = 0.1$)

- Faster convergence
- Stable training
- Better prediction accuracy

Thus, selecting an appropriate learning rate improves convergence and model performance.

Influence of Activation Functions

Activation functions introduce non-linearity into the neural network.

Sigmoid Function

$$\text{Output} = 1 / (1 + e^{(-x)})$$

- Output lies between 0 and 1.
- Suitable for binary classification.
- Produces probability values.

ReLU Function

$$\text{ReLU}(x) = \max(0, x)$$

- Faster training.

- Reduces the vanishing gradient problem.
- Commonly used in hidden layers.

For binary classification, the output layer usually uses the **Sigmoid** activation function because it predicts probabilities between 0 and 1.

Performance Evaluation Using the Dataset

Using the above dataset,

Positive Class = **Yes**

Negative Class = **No**

Calculation of TP, TN, FP and FN

True Positive (TP)

Actual = Yes and Predicted = Yes

Examples:

- Example 1
- Example 4

TP = 2

True Negative (TN)

Actual = No and Predicted = No

Example:

- Example 3

TN = 1

False Positive (FP)

Actual = No but Predicted = Yes

Example:

- Example 5

FP = 1

False Negative (FN)

Actual = Yes but Predicted = No

Example:

- Example 2

FN = 1

Confusion Matrix

Predicted Yes Predicted No

Actual Yes TP = 2 FN = 1

Actual No FP = 1 TN = 1

Evaluation Metrics

1. Accuracy

Formula

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Calculation

Accuracy

$$= (2 + 1) / (2 + 1 + 1 + 1)$$

$$= 3 / 5$$

$$= 0.60$$

$$= 60\%$$

2. Precision

Formula

$$\text{Precision} = TP / (TP + FP)$$

Calculation

Precision

$$= 2 / (2 + 1)$$

$$= 2 / 3$$

$$= 0.667$$

$$= 66.7\%$$

3. Recall (Sensitivity)

Formula

$$\text{Recall} = TP / (TP + FN)$$

Calculation

Recall

$$= 2 / (2 + 1)$$

$$= 2 / 3$$

$$= 0.667$$

$$= 66.7\%$$

4. F1-Score

Formula

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Calculation

F1 Score

$$= 2 \times (0.667 \times 0.667) / (0.667 + 0.667)$$

$$= 0.667$$

$$= 66.7\%$$

Performance Analysis

The confusion matrix shows:

- True Positives = **2**
- True Negatives = **1**
- False Positives = **1**
- False Negatives = **1**

The neural network correctly classified **3 out of 5** students, resulting in an **accuracy of 60%**. The **precision of 66.7%** indicates that two-thirds of the students predicted as placed were actually placed. The **recall of 66.7%** shows that the model successfully identified two-thirds of the actual placed students. The **F1-score of 66.7%** provides a balanced measure of precision and recall. Although the model performs reasonably well, reducing false positives and false negatives would further improve its reliability and prediction accuracy.

Role of Data Preprocessing

Data preprocessing prepares the raw data before it is used for training the neural network.

The preprocessing steps include:

- Removing duplicate records.
- Handling missing values.
- Encoding categorical values such as High, Medium, and Low into numerical values.
- Normalising aptitude scores so that all features have a similar scale.
- Removing noisy or inconsistent data.

These steps improve the quality of the training data, reduce errors, increase training speed, and improve the overall prediction accuracy of the neural network.

Role of Feature Selection

Feature selection is the process of selecting only the most important features for model training.

For this dataset, the important features are:

- Aptitude Score
- Communication Skill
- Programming Skill

Selecting only the relevant features provides several advantages:

- Removes unnecessary information.
- Reduces overfitting.
- Decreases computational complexity.
- Speeds up model training.
- Improves prediction accuracy.
- Makes the neural network more reliable.

Conclusion

The Multilayer Perceptron (MLP) uses the backpropagation algorithm to learn from training data by repeatedly updating the network weights based on the prediction error. During the forward pass, the input features are processed through the hidden layer to generate a predicted output. The difference between the predicted and actual outputs is calculated using the Mean Squared Error (MSE), and the error is propagated backwards to update the weights. A suitable learning rate ensures stable convergence, while activation functions such as Sigmoid and ReLU enable the network to learn complex non-linear relationships. Using the given dataset, the confusion matrix produced TP = 2, TN = 1, FP = 1, and FN = 1, resulting in an accuracy of 60%, precision of 66.7%, recall of 66.7%, and F1-score of 66.7%. Furthermore, effective data preprocessing and feature selection improve the model's accuracy, efficiency, and reliability by providing clean, relevant, and high-quality input data. This demonstrates how backpropagation enables an MLP to achieve accurate and reliable binary classification.

2. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution. Explain how selection chooses the best individuals, how crossover generates new offspring, and how mutation helps in exploring the hypothesis space. Also, provide the final optimized solution after evolution.

2.Sol:- Genetic Algorithm for Optimizing Candidate Solutions

Introduction

A **Genetic Algorithm (GA)** is an optimization and search technique inspired by the process of natural selection. It is used to find the best solution among many candidate solutions by applying operations such as **selection**, **crossover**, and **mutation**. In each generation, the population evolves, and individuals with higher fitness have a greater chance of producing better offspring. The process continues until the best or optimal solution is obtained.

Dataset: Candidate Solutions for Speed and Accuracy

The following dataset represents different candidate solutions evaluated based on **Speed** and **Accuracy**. The objective is to identify the most suitable solution.

Candidate	Speed	Accuracy	Suitable (Target)
A	70	80	Yes
B	60	65	No
C	85	75	Yes
D	50	55	No
E	90	95	Yes

Fitness Function

The fitness value is calculated as:

$$\text{Fitness} = (\text{Speed} + \text{Accuracy}) / 2$$

Step 1: Initial Population

Calculate the fitness value for each candidate.

Candidate	Speed	Accuracy	Fitness
A	70	80	$(70 + 80)/2 = 75$
B	60	65	$(60 + 65)/2 = 62.5$
C	85	75	$(85 + 75)/2 = 80$
D	50	55	$(50 + 55)/2 = 52.5$

E	90	95	$(90 + 95)/2 = 92.5$
---	----	----	----------------------

The candidate with the highest fitness is **E (92.5)**.

Step 2: Selection

Selection chooses the best individuals from the population to become parents for the next generation.

The two candidates with the highest fitness are:

Parent Fitness

E 92.5

C 80

These two candidates are selected because they have better fitness values than the others.

Step 3: Crossover

Crossover combines the characteristics of the selected parents to produce new offspring.

Parents

Parent 1 (E)

Speed = 90

Accuracy = 95

Parent 2 (C)

Speed = 85

Accuracy = 75

Offspring after Crossover

Offspring Speed Accuracy

Child 1 90 75

Child 2 85 95

Fitness of Offspring

Child 1

Fitness = $(90 + 75)/2$

= 82.5

Child 2

Fitness = $(85 + 95)/2$

= 90

The offspring inherit useful characteristics from both parents.

Step 4: Mutation

Mutation introduces a small random change in one of the offspring to increase diversity and avoid getting stuck at a local optimum.

Suppose Child 2 undergoes mutation.

Before mutation

Speed = 85

Accuracy = 95

After mutation

Speed = 88

Accuracy = 95

New Fitness

$\text{Fitness} = (88 + 95)/2$

= 91.5

Mutation slightly improves the offspring's fitness and helps explore new possible solutions.

Population Evolution

Generation 1

Candidate Fitness

A	75
B	62.5
C	80
D	52.5
E	92.5

Best Candidate = E

Generation 2 (After Selection and Crossover)

Candidate Fitness

Child 1	82.5
Child 2	90

Best Candidate = **Child 2**

Generation 3 (After Mutation)

Candidate	Fitness
-----------	---------

Mutated Child	91.5
---------------	------

The population gradually improves over generations.

Best Optimal Solution

The highest fitness value is:

Candidate	Speed	Accuracy	Fitness
-----------	-------	----------	---------

E	90	95	92.5
---	----	----	------

Therefore, **Candidate E** is selected as the **final optimized solution** because it has the highest fitness value.

Role of Selection

Selection chooses the fittest individuals from the current population based on their fitness values. Better individuals have a higher probability of being selected for reproduction. This ensures that good characteristics are passed to the next generation, leading to improved solutions over time.

Role of Crossover

Crossover combines the characteristics of two selected parents to create new offspring. The offspring inherit useful features from both parents, increasing the possibility of obtaining better solutions than the original population. It improves exploration of the search space and accelerates convergence toward the optimal solution.

Role of Mutation

Mutation introduces small random changes in the offspring's genes. It prevents the algorithm from converging too early to a local optimum and increases the diversity of the population. Mutation enables the Genetic Algorithm to explore new areas of the hypothesis space and discover better solutions that may not be obtained through selection and crossover alone.

Improvement Across Generations

Generation	Best Candidate	Fitness
------------	----------------	---------

Generation 1	E	92.5
--------------	---	------

Generation 2	Child 2	90
--------------	---------	----

Generation 3	Mutated Child	91.5
--------------	---------------	------

The Genetic Algorithm improves the population by repeatedly selecting the fittest individuals, producing new offspring through crossover, and introducing diversity through

mutation. As the generations progress, weaker solutions are replaced by stronger ones, resulting in better overall performance.

Advantages of Genetic Algorithms

- Finds near-optimal solutions efficiently.
- Handles large and complex search spaces.
- Avoids getting trapped in local optima.
- Does not require gradient information.
- Can solve optimization problems with multiple variables.
- Produces better solutions through continuous evolution.

Conclusion

In this example, a Genetic Algorithm was applied to optimize candidate solutions based on Speed and Accuracy. The initial population was evaluated using a fitness function, and the best candidates were selected for reproduction. Selection chose the fittest individuals, crossover combined their characteristics to generate new offspring, and mutation introduced small random changes to increase diversity and explore new solutions. Through successive generations, the population evolved toward better-performing candidates. Based on the fitness values, Candidate E (Speed = 90, Accuracy = 95, Fitness = 92.5) was identified as the final optimized solution, demonstrating how Genetic Algorithms effectively solve optimization problems by mimicking the process of natural evolution.

3. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs.

3.Sol:- Neural Network for Disease Classification using Backpropagation

Introduction

A **Neural Network** is a machine learning model inspired by the human brain. It consists of an **input layer**, one or more **hidden layers**, and an **output layer**. Neural networks are widely used in healthcare to predict diseases based on patient information. In this problem, the objective is to predict whether a patient has a disease (**Yes**) or does not have a disease (**No**) using attributes such as **Age, Blood Pressure, Glucose Level, and Lifestyle Risk**.

The **Backpropagation algorithm** is used to train the neural network by calculating the prediction error and updating the weights to improve the model's accuracy.

Dataset: Disease Prediction

Patient	Age	Blood Pressure (mmHg)	Glucose Level (mg/dL)	Lifestyle Risk	Actual Disease	Predicted Disease
P1	45	150	180	High	Yes	Yes
P2	32	120	95	Low	No	No
P3	60	160	200	High	Yes	No
P4	28	118	100	Low	No	Yes
P5	52	145	170	Medium	Yes	Yes

Input Features

- Age
- Blood Pressure
- Glucose Level
- Lifestyle Risk

Output

- **Yes (1)** = Disease Present
- **No (0)** = Disease Not Present

For explanation, consider **Patient P3**.

- Age = **60**

- Blood Pressure = **160 mmHg**
- Glucose Level = **200 mg/dL**
- Lifestyle Risk = **High**
- Actual Output = **Yes (1)**

Architecture of the Neural Network

Input Layer

Age

Blood Pressure

Glucose Level

Lifestyle Risk



Hidden Layer

(H1 and H2)



Output Layer

Disease (Yes / No)

Step 1: Forward Pass

Initially, random weights are assigned to all connections.

Initial Weights

Input → Hidden Layer

$$w1 = 0.30$$

$$w2 = 0.40$$

$$w3 = 0.20$$

$$w4 = 0.50$$

Hidden → Output Layer

$$w5 = 0.60$$

$$w6 = 0.70$$

Each hidden neuron computes a weighted sum.

Weighted Sum

$$Z = (x1 \times w1) + (x2 \times w2) + (x3 \times w3) + (x4 \times w4) + b$$

where,

- x_1 = Age
- x_2 = Blood Pressure
- x_3 = Glucose Level
- x_4 = Lifestyle Risk
- w = Weight
- b = Bias

The weighted sum is passed through the Sigmoid activation function.

Sigmoid Function

$$\text{Output} = 1 / (1 + e^{(-Z)})$$

Suppose the hidden layer produces

Hidden Neuron H1 = 0.80

Hidden Neuron H2 = 0.85

The output neuron predicts

Predicted Output = 0.68

The actual output is

Target = 1

Since the prediction differs from the target, an error is generated.

Step 2: Error Calculation

The prediction error is calculated as

$$\text{Error} = \text{Target} - \text{Predicted Output}$$

Substituting the values,

$$\text{Error} = 1 - 0.68$$

$$\text{Error} = 0.32$$

The Mean Squared Error (MSE) is

$$\text{MSE} = (1/2) \times (\text{Target} - \text{Predicted Output})^2$$

Calculation

$$\text{MSE} = (1/2) \times (1 - 0.68)^2$$

$$= (1/2) \times (0.32)^2$$

$$= (1/2) \times 0.1024$$

$$= 0.0512$$

The objective is to reduce this error during training.

Step 3: Backpropagation

The error is propagated backwards through the network, and the weights are updated.

Weight Update Formula

$$\text{New Weight} = \text{Old Weight} + (\text{Learning Rate} \times \text{Error} \times \text{Input})$$

or

$$w(\text{new}) = w(\text{old}) + \eta \times \text{Error} \times \text{Input}$$

where

- η = Learning Rate

Assume

$$\text{Old Weight} = 0.60$$

$$\text{Learning Rate} = 0.10$$

$$\text{Input} = 0.80$$

$$\text{Error} = 0.32$$

Calculation

New Weight

$$= 0.60 + (0.10 \times 0.32 \times 0.80)$$

$$= 0.60 + 0.0256$$

$$= 0.6256$$

$$\approx 0.626$$

The updated weights reduce the prediction error in the next training iteration.

Comparison of Actual and Predicted Outputs

Patient	Actual Disease	Predicted Disease
P1	Yes	Yes
P2	No	No
P3	Yes	No
P4	No	Yes
P5	Yes	Yes

Calculation of TP, TN, FP and FN

True Positive (TP)

Actual = Yes and Predicted = Yes

- P1
- P5

$$TP = 2$$

True Negative (TN)

Actual = No and Predicted = No

- P2

$$TN = 1$$

False Positive (FP)

Actual = No but Predicted = Yes

- P4

$$FP = 1$$

False Negative (FN)

Actual = Yes but Predicted = No

- P3

$$FN = 1$$

Confusion Matrix

	Predicted Yes	Predicted No
Actual Yes	TP = 2	FN = 1
Actual No	FP = 1	TN = 1

Evaluation Metrics

1. Accuracy

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Calculation

Accuracy

$$= (2 + 1) / (2 + 1 + 1 + 1)$$

$$= 3 / 5$$

$$= 0.60$$

$$= 60\%$$

2. Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Calculation

Precision

$$= 2 / (2 + 1)$$

$$= 2 / 3$$

$$= 0.667$$

$$= 66.7\%$$

3. Recall (Sensitivity)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Calculation

Recall

$$= 2 / (2 + 1)$$

$$= 2 / 3$$

$$= 0.667$$

$$= 66.7\%$$

4. F1-Score

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Calculation

F1 Score

$$= 2 \times (0.667 \times 0.667) / (0.667 + 0.667)$$

$$= 0.667$$

$$= 66.7\%$$

Performance Analysis

From the confusion matrix:

- **True Positives (TP) = 2**
- **True Negatives (TN) = 1**
- **False Positives (FP) = 1**
- **False Negatives (FN) = 1**

The neural network correctly classified **3 out of 5 patients**, resulting in an **accuracy of 60%**. The **precision of 66.7%** indicates that two-thirds of the patients predicted to have the disease actually had the disease. The **recall of 66.7%** shows that the model correctly identified two-thirds of the patients who truly had the disease. The **F1-score of 66.7%** provides a balanced measure of precision and recall. Reducing false positives and false negatives would further improve the reliability of the model.

Advantages of Backpropagation

- Reduces prediction error by updating weights.
- Improves classification accuracy.
- Learns complex relationships between patient attributes.
- Supports reliable medical diagnosis.
- Works efficiently with multilayer neural networks.

Conclusion

In this example, a Multilayer Perceptron (MLP) was used to classify whether a patient has a disease based on Age, Blood Pressure, Glucose Level, and Lifestyle Risk. During the forward pass, the input features were processed through the hidden layer to produce a predicted output. The prediction error was calculated using the Mean Squared Error (MSE), and the backpropagation algorithm updated the weights to minimise this error. The model's performance was evaluated using a confusion matrix, where $TP = 2$, $TN = 1$, $FP = 1$, and $FN = 1$. The evaluation metrics showed an accuracy of 60%, precision of 66.7%, recall of 66.7%, and F1-score of 66.7%. These results demonstrate how neural networks and backpropagation can effectively perform disease classification and continuously improve prediction accuracy through training.

